

CHROMA: High-Quality Robust Graph Coloring on GPUs

Chou-Ying Hsieh^{*†}, Chien-Hsien Wu^{*}, Sy-Yen Kuo^{*‡}

^{*}Department of Electrical Engineering, National Taiwan University, Taipei, Taiwan

[†]Department of Electrical and Computer Engineering, University of Wisconsin-Madison, WI, USA

[‡]Department of Computer Science and Information Engineering, Chang Gung University, Taoyuan, Taiwan
{f07921043, r12k41022, sykuo}@ntu.edu.tw

Abstract—Graph coloring assigns colors to vertices such that no two adjacent vertices share the same color. As graphs grow in size and complexity, scalable graph coloring becomes increasingly challenging. Although many CUDA-based parallel coloring algorithms have been proposed, they often face a quality–performance trade-off: strict priority orderings can reduce the number of colors but limit parallelism. We present CHROMA, a high-quality, efficient, and robust multi-GPU graph coloring framework that consolidates the smallest-degree-first priority-allocation stages in the Jones–Plassmann (JP) algorithm to reduce synchronization overhead. Building on this foundation, CHROMA introduces three optimizations—efficient loop scanning, smallest-degree calibration, and elastic granularity control—to improve coloring quality, reduce runtime, and provide tunable control over the trade-off. CHROMA further integrates a robust graph partitioning strategy to scale across multiple GPUs. Experiments on diverse real-world graphs show that CHROMA achieves up to a $7\times$ geometric-mean speedup on a single GPU over state-of-the-art methods while maintaining competitive coloring quality. We open-source CHROMA to facilitate future research.

Index Terms—graph coloring, parallel computing, GPU, multi-GPU, ordering heuristics, Jones–Plassmann algorithm.

I. INTRODUCTION

Graph coloring is a fundamental NP-hard problem in graph theory [1]. It assigns colors to vertices such that no two adjacent vertices share the same color, with the objective of minimizing the total number of colors. Graph coloring has broad applications in both theoretical and practical settings, including resource allocation [2], and integrated circuit design [3], where effective partitioning is critical for performance. In scientific computing, graph coloring also supports sparse Jacobian optimization [4] and LU decomposition [5].

Despite its importance, the increasing size and complexity of real-world graphs pose significant scalability challenges. To meet these demands, researchers have leveraged parallel and distributed computing, particularly on graphics processing units (GPUs). Existing parallel graph coloring algorithms largely fall into two families: speculative greedy (SGR) [6], [7] and maximal independent set (MIS)-based [8] approaches. Compared with SGR, MIS-based methods typically incur fewer conflicts and require less synchronization, making them well suited for GPU execution.

Luby’s algorithm [8] is an early parallel MIS-based graph coloring method in which vertices with the highest local pri-

ority are colored simultaneously. Building on this idea, Jones and Plassmann (JP) [9] proposed an alternative that avoids recomputing independent sets at each iteration. In JP-based graph coloring, the algorithm first assigns a global priority ordering to all vertices (the **Priority Allocation (PA)** phase) and then colors vertices in parallel while resolving conflicts (the **Color Allocation (CA)** phase). This design preserves coloring quality while enabling substantial parallelism.

To improve coloring quality, JP-based methods often employ heuristic priority orderings in the PA phase [10], including First-Fit (FF, input order), Random (R), Largest-Degree-First (LF), Smallest-Last (SL), Incidence-Degree (ID, most colored neighbors; ties broken by degree), and Saturation-Degree (SD, most distinct neighbor colors; ties broken by degree). Higher-quality orderings typically reduce conflicts, but their effectiveness is graph dependent. As reported in [10], their coloring quality follows:

$$FF < R < LF < ID \leq SL < SD.$$

However, higher-quality orderings are often inherently sequential or rely on strict selection rules, which increases the cost of priority allocation. For example, JP-SD yields the best quality among these heuristics, but it effectively forces a sequential coloring order; JP-SL (the second-best) iteratively selects and removes minimum-degree vertices. These characteristics make the PA phase time-consuming. Moreover, the sequential nature of these high-quality JP variants makes them difficult to distribute for additional acceleration on large-scale graphs. (Throughout the paper, we use a suffix (e.g., -SD) to denote the corresponding JP variant.)

To address these challenges while preserving high coloring quality, we propose **CHROMA**¹, a high-quality robust graph coloring framework on GPUs. CHROMA redesigns the priority allocation procedure of JP-SL to be parallel and robust on single- and multi-GPU systems, without sacrificing coloring quality. We summarize the contributions of CHROMA as follows:

- We propose **CHROMA**, a robust and scalable GPU-based JP-SL framework for parallel and distributed graph coloring that balances quality and runtime efficiency.

¹GitHub: <https://github.com/NTUDDSNLab/CHROMA>

Algorithm 1: Jones and Plassmann Algorithm

Data: Undirected Graph: $G = (V, E)$,
Priority assignment: $p : V \rightarrow \mathbb{N}$,
Candidate color sequence set:
 $\mathcal{Q} = \{Q_v \leftarrow [1, 1, \dots, 1] \text{ of length } D(v) + 1 \mid v \in V\}$
Result: Color assignment $c : V \rightarrow \mathbb{N}$

```
/* Stage 1: Priority Allocation (PA) */
1 forall v in V do
2   p(v) ← OrderingHeuristic(v);
/* Stage 2: Color Allocation (CA) */
3 repeat
4   forall v in V such that c(v) is NULL in parallel do
5     if v has the biggest p(v) among N(v) then
6       c(v) ← ColorSelect(Q_v);
7 until all v in V are colored;
```

- We design **cuSL**, the first GPU-based parallel priority allocation scheme for JP-SL.
- Building on cuSL, we introduce three optimizations: **Efficient Loop Scanning (ELS)**, **Smallest Degree Calibration (SDC)**, and **Elastic Granularity Control (EGC)**. ELS improves loop scanning efficiency and increases cuSL utilization. SDC improves coloring quality by dynamically updating the minimum degree for the next iteration. EGC exposes a tunable parameter to balance coloring quality and runtime. We also develop a **learning-based model** that predicts the optimal EGC parameter for a given graph, guiding users to an effective trade-off configuration in CHROMA.
- We present a **Robust Graph Partitioning (RGP)** module for multi-GPU execution. RGP is partitioner-agnostic and uses ghost vertices and pre-colored boundaries to maintain global consistency across partitions.
- Through extensive experiments on large-scale graphs, we demonstrate that CHROMA outperforms existing methods in both coloring quality and runtime, while scaling effectively across multiple GPUs.

The remainder of the paper is organized as follows: Section II reviews the JP algorithm and the SL heuristic. Section III discusses related work. Section IV details CHROMA’s four core optimizations. Section V details the prediction model for EGC. Experimental results are presented in Section VI. Section VII concludes the paper.

II. BACKGROUND

This section first reviews the Jones and Plassmann (JP) algorithm and its overall workflow. We then explain, using preliminary experiments, why the priority allocation phase of JP-SL is time-consuming. Finally, we summarize the GPU architecture and execution model relevant to our design.

A. Jones and Plassmann Algorithm

Algorithm 1 illustrates the general JP framework. Given an undirected graph $G = (V, E)$, the goal is to compute a

Algorithm 2: Smallest-Last (SL) Ordering Heuristic

Data: Graph $G = (V, E)$
Result: Priority assignment $p : V \rightarrow \mathbb{N}$

```
1 L ← ∅ ;
2 G' ← G ;
3 M ← ∞
4 while G' is not empty do
5   Scan all vertices to update M ;
6   Append vertex v with degree M to L ;
7   Remove v from G'
8 Reverse(L) ;
9 foreach v in L do
10  Assign unique priority p(v) based on order in L
```

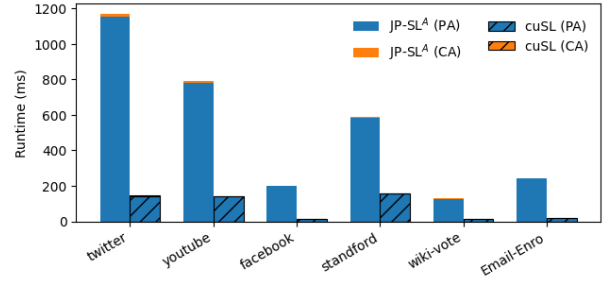


Fig. 1: Runtime breakdown of JP-SL^A running on 192 threads Intel Xeon server compared to cuSL running on an NVIDIA RTX 4090 GPU.

coloring $c : V \rightarrow \mathbb{N}$ that assigns a color to each vertex. JP maintains a priority function $p : V \rightarrow \mathbb{N}$ and, for each vertex v , a candidate color set Q_v that represents currently available colors. In our implementation, Q_v is encoded as a binary vector of length $D(v) + 1$, where $D(v)$ denotes the degree of v (the pessimistic upper bound on the number of colors needed for v).

The JP algorithm consists of two stages: priority allocation (PA) and color allocation (CA). In PA (lines 1–2), an ordering heuristic assigns a priority to each vertex. In CA (lines 3–8), vertices are colored in parallel; a vertex v may color itself when it has the highest priority among its neighbors $N(v)$. The selected color is chosen from the available set Q_v .

B. Priority Allocation of JP-SL

Algorithm 2 shows the priority allocation of JP-SL proposed by Mutala *et al.* [11]. At each iteration, the algorithm scans the current graph G' to find the minimum degree M (line 5), appends a vertex v with $\deg(v) = M$ to a list L (line 6), and removes v from G' (line 7). This process repeats until G' becomes empty, after which reversing L yields the final priority ordering. As expected, this procedure is inherently sequential and typically requires many iterations on large graphs. To parallelize JP-SL, Allwright *et al.* [12] relax the PA rule: instead of removing a single vertex per iteration, they remove all vertices with the same minimum degree before

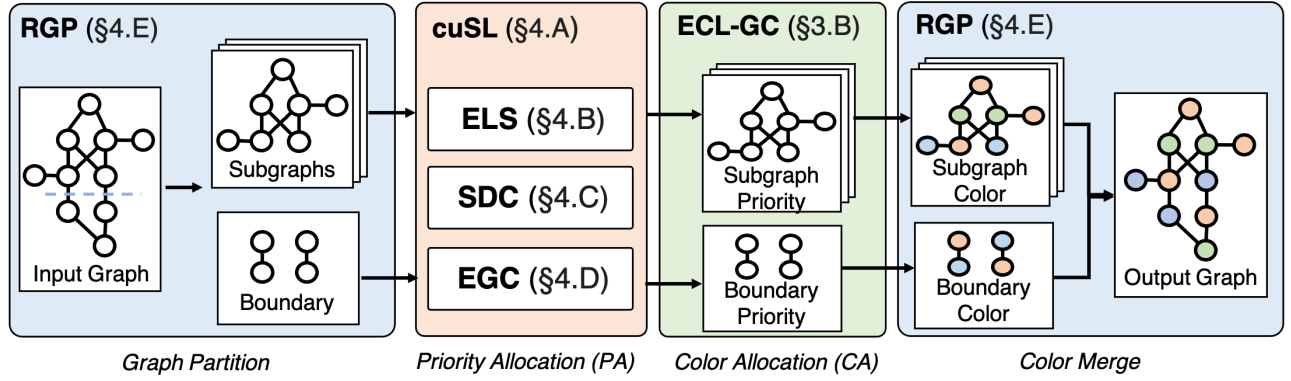


Fig. 2: CHROMA architecture. CHROMA supports execution on single- and multi-GPU systems. If CHROMA uses single-GPU, it will only execute the priority allocation and color allocation phases.

updating remaining degrees. This modification substantially reduces the number of iterations, but can slightly degrade coloring quality. Because both variants are referred to as JP-SL in the literature, we denote them as JP-SL^M and JP-SL^A, respectively.

Fig. 1 reports the runtime breakdown of JP-SL^A running on 192 threads of a dual-socket 4th Gen Intel Xeon Scalable processors compared with cuSL running on an NVIDIA RTX 4090 GPU. In these preliminary experiments, we observe that priority allocation (PA) is the bottleneck in JP-SL^A on both CPU and GPU designs: most of the runtime is spent in PA, which can take up to 100× longer than color allocation (CA) on CPU. This motivates us to parallelize the PA phase on GPUs. These results also suggest that cuSL can achieve substantial speedups on GPUs.

C. GPU Architecture and Execution Model

For clarity, we use NVIDIA Compute Unified Device Architecture (CUDA) terminology throughout the remainder of the paper. GPUs follow the SIMT (single-instruction, multiple-thread) execution model, in which many threads execute the same instruction stream on different data. Conceptually, a kernel launch forms a *grid* of thread *blocks* scheduled on streaming multiprocessors (SMs). Each block (cooperative thread array, CTA) is further partitioned into *warps*, the hardware execution unit of 32 threads that share a program counter and execute in lockstep. CUDA provides *warp primitives* for efficient intra-warp communication and synchronization, which are particularly important for irregular workloads such as graph coloring.

III. RELATED WORKS

A. Approximate Ordering Heuristic

Prior work has sought to accelerate JP-based approaches by reducing the iterations in the priority-allocation (PA) phase of JP-SL. JP-SLL [10] discards any neighbor whose degree is below 2^d , instead of removing the minimum-degree vertex (Algorithm 2, Line 4). A user-defined parameter r bounds the number of iterations performed per threshold d , thereby

controlling granularity. JP-ADG [13] removes vertices whose degrees fall below the average degree of the remaining graph, substantially reducing iterations in PA but generally worsening coloring quality relative to conventional smallest-last (SL). Overall, these heuristics trade coloring quality for speed by approximating the SL removal order. In contrast, CHROMA directly parallelizes SL-based PA on GPUs and introduces a robust parallel ordering (cuSL) parameterized by θ to continuously tune removal granularity, offering finer control than JP-SLL or JP-ADG.

B. Optimizations on Color Allocation

ECL-GC [14], [15] introduces a highly efficient CUDA-based kernel for the color allocation (CA) phase. The *priority directed acyclic graph (DAG)* converts the input graph into a DAG based on vertex priorities in parallel. During coloring, each vertex checks whether all its in-neighbors—those with higher priority—have already been colored. Once this condition is met, the vertex colors itself immediately, eliminating the need for frequent global synchronization. The *shortcut* technique enables a vertex to proceed with coloring early if none of its neighbors have selected its preferred color. ECL-GC-R [15] further introduces a *post-processing color reduction* technique to refine the coloring quality after CA stage.

Although ECL-GC adopts the largest-degree-first ordering heuristic, these optimizations are orthogonal to cuSL. As such, CHROMA incorporates both techniques to improve color allocation’s efficiency while maintaining compatibility with its custom priority allocation strategy.

IV. METHODOLOGIES

Fig. 2 summarizes the CHROMA workflow, which consists of four phases: graph partitioning, priority allocation, color allocation, and color merging. The Robust Graph Partitioning (RGP) module partitions the input graph into subgraphs and a boundary graph to support the final merging step. At the core of CHROMA, cuSL implements a CUDA-based SL ordering heuristic to assign priorities for coloring, enhanced by three optimizations: Efficient Loop Scanning (ELS) to reduce scanning overhead via loop fusion and warp-level parallelism,

Algorithm 3: cuSL Ordering Heuristic on GPU

Data: Graph $G = (V, E)$ **Result:** Priority list $p[|V|]$

```
1  $n \leftarrow 0$  ;
2  $M \leftarrow \infty$  ;
3  $p[] \leftarrow \{-1\}$  ;
4 for  $iter = 0$ ;  $n < |V|$ ;  $iter \leftarrow iter + 1$  do
5   foreach  $v \in V$  such that  $p[v] == -1$  do
6     if  $d[v] < M$  then
7        $M \leftarrow d[v]$  ;
8   foreach
9      $v \in V$  such that  $p[v] == -1$  and  $d[v] == M$  do
10     $p[v] \leftarrow iter$  ;
11     $n \leftarrow n + 1$  ;
12    foreach  $u \in N(v)$  do
13       $d[u] \leftarrow d[u] - 1$  ;
14   $M \leftarrow \infty$  ;
```

Smallest Degree Calibration (SDC) to improve coloring quality, and Elastic Granularity Control (EGC) to expose a tunable trade-off between coloring quality and execution time.

A. Parallel Smallest-Last Ordering on CUDA (cuSL)

We design cuSL, the first GPU implementation of SL^A ordering. We reformulate Algorithm 2 into a GPU-friendly version, shown in Algorithm 3. We represent the input graph G using the compressed sparse row (CSR) format, a common and memory-efficient representation on GPUs. Because removing vertices from a CSR graph is impractical, we maintain a priority array $p[]$ initialized to -1 to mark vertices that have not yet been assigned priorities. Using this representation, cuSL finds the current minimum degree by striding over vertices with $p[v] == -1$ and tracking the minimum of $d[v]$ in parallel. After determining the minimum degree, cuSL assigns priority $iter$ to vertices that satisfy the minimum-degree criterion and decrements the degrees of their neighbors $N(v)$ (Algorithm 3, lines 8–12). This procedure repeats until all vertices are assigned priorities; we use n to count the number of assigned vertices and $iter$ to denote the iteration index. In a straightforward implementation, each loop maps one thread to one vertex (Algorithm 3, lines 5 and 8) and requires a global synchronization point between the two loops to ensure correctness.

B. Efficient Loop Scanning (ELS)

Algorithm 3 exposes two major optimization opportunities: (i) redundant scanning and (ii) underutilized parallelism. Algorithm 4 presents cuSL with **Efficient Loop Scanning (ELS)**. In the baseline, cuSL uses two separate passes: one to find the current minimum degree and another to assign priorities to minimum-degree vertices, which scans the vertex array twice per iteration and increases global-memory traffic. ELS integrates these passes into a single traversal by maintaining

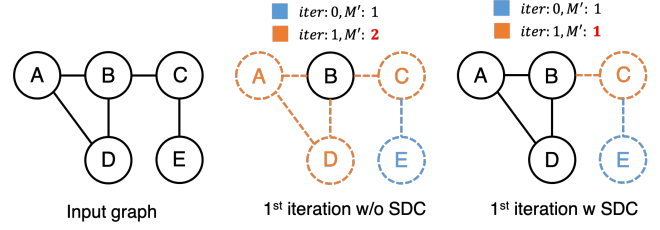


Fig. 3: The difference of the next-iteration minimum degree M' with and without SDC in the first two iterations. We assume each thread is responsible for a vertex.

Algorithm 4: cuSL with ELS

```
1  $W \leftarrow \emptyset$  ;
2  $M' \leftarrow \infty$  ;
3  $M \leftarrow 1$  ;
4 for  $iter = 0$ ;  $n < |V|$ ;  $iter \leftarrow iter + 1$  do
5   /* Loop integration */
6   foreach  $v \in V$  such that  $p[v] == -1$  do
7     if  $d[v] \leq M$  then
8        $p[v] \leftarrow iter$  ;
9        $W \leftarrow W \cup v$  ;
10    else
11       $M' \leftarrow \min(M', d[v])$  ;
12  /* Warp-based parallel removal */
13  foreach  $v \in W$  do
14    foreach  $u \in N(v)$  do
15       $d[u] \leftarrow d[u] - 1$  ;
16   $n \leftarrow n + |W|$  ;
17   $W \leftarrow \emptyset$  ;
18   $M \leftarrow M'$  ;
19   $M' \leftarrow \infty$  ;
```

two variables: M (the current iteration's minimum-degree threshold) and M' (a candidate minimum degree for the next iteration). During the scan, cuSL assigns priorities to vertices satisfying $d[v] \leq M$ and simultaneously updates M' for the next iteration in the **else** branch. Without loss of generality, we initialize $M = 1$ since isolated vertices ($d[v] = 0$) can be prioritized without conflicts.

Loop integration also enables a second-level parallel optimization. Because ELS identifies all vertices selected in the current iteration, it can aggregate them into a worklist W and update neighbor degrees using warp-level parallelism, rather than processing neighbors sequentially. We evaluate the impact of ELS in Section VI-C.

C. Smallest Degree Calibration (SDC)

Loop integration can overestimate the next-iteration minimum degree M' because neighbor-degree updates occur after the scan that records M' . To address this issue, we propose Smallest Degree Calibration (SDC), shown in Algorithm 5, which updates M' during neighbor-degree decrements. Fig. 3

Algorithm 5: Smallest Degree Calibration (SDC) for cuSL with ELS. A modification of Algorithm 4 line: 11-13.

```

1 foreach  $v \in W$  do
2   foreach  $u \in N(v)$  do
3      $d[u] \leftarrow d[u] - 1$  ;
4     /* Smallest Degree Calibration (SDC) */
5     if  $d[u] < M'$  then
6        $M' \leftarrow d[u]$  ;

```

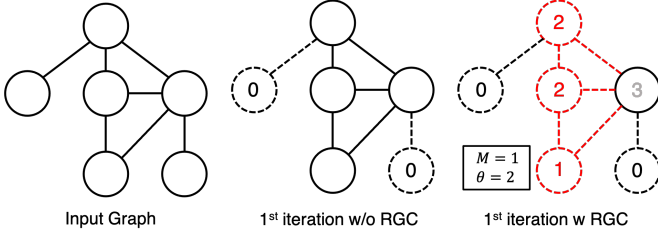


Fig. 4: The priority allocation with and without elastic granularity control (EGC) in an iteration. Assuming $M = 1$ and elastic parameter $\theta = 2$. The number inside a vertex represents its priority.

illustrates the effect. In the first iteration ($iter = 0, M = 1$), thread E assigns a priority to vertex E and decreases vertex C's degree from 2 to 1. With SDC, thread E observes the updated degree and sets $M' = 1$ accordingly. Without SDC, if thread C records its local minimum before thread E performs the decrement, the global minimum for the next iteration may be overestimated as 2 (middle sub-figure in Fig. 3). By calibrating M' during updates, SDC yields a more accurate next-iteration minimum degree, which results in a more faithful SL ordering. We validate SDC via comprehensive experiments in Section VI.

D. Elastic Granularity Control (EGC)

As the priority allocation proceeds, the number of iterations required to identify minimum-degree vertices can become large, increasing runtime. Inspired by the approximate ordering technique in Section III-A, CHROMA introduces a tunable elastic parameter θ to control the removal granularity. As shown in Algorithm 6, CHROMA relaxes the original selection rule from $d[v] \leq M$ (the minimum degree) to $d[v] \leq M + \theta$. Elastic Granularity Control (EGC) therefore provides a controllable trade-off between runtime and coloring quality. When $\theta = 0$, CHROMA matches the ordering quality of JP-SL^A. When $\theta > 0$, the eligible degree range per iteration increases, reducing the number of iterations. For example, if $M = 3$ and $\theta = 3$, threads may assign priorities to vertices with degrees up to $M + \theta = 6$ in the same iteration.

However, selecting a broader set of vertices per iteration can degrade color quality if multiple vertices are assigned identical priorities. To mitigate this effect, EGC incorporates each

Algorithm 6: Elastic Granularity Control (EGC) for cuSL with ELS. A modification of Algorithm 4 line: 5-10.

```

1 foreach  $v \in V$  such that  $p[v] == -1$  do
2   if  $d[v] \leq M + \theta$  then
3      $p[v] \leftarrow iter + (d[v] - M)$  ;
4      $W \leftarrow W \cup v$  ;
5   else
6      $M' \leftarrow \min(M', d[v])$  ;
7  $iter \leftarrow iter + \theta$  ;

```

Algorithm 7: Robust Graph Partition (RGP) procedure.

Data: Input Graph $G = (V, E)$
Result: Color assignment $c : V \rightarrow \mathbb{N}$

```

1  $\mathcal{G}, b \leftarrow \text{Partitioner}(G)$  ;
2  $b_p \leftarrow \text{PriorityAllocator}(b)$  ;
3  $b_c \leftarrow \text{ColorAllocator}(b_p)$  ;
4 foreach  $g \in \mathcal{G}$  do
5    $g_p \leftarrow \text{PriorityAllocator}(g)$  ;
6    $g_c \leftarrow \text{ColorAllocator}(g_p + b_c)$  ;
7    $c \leftarrow \text{ColorMerge}(g_c)$  ;

```

selected vertex's degree into its assigned priority (Algorithm 6, line 3), inducing a localized degree-based ordering among the selected vertices. For instance, with $M = 3, \theta = 3$, and $iter = 2$, two vertices v_i and v_j with degrees 4 and 5 can be assigned distinct priorities $p[i] = 4$ and $p[j] = 5$, respectively, instead of the same value.

Fig. 4 contrasts the first iteration with and without EGC. Without EGC, CHROMA selects only the minimum-degree vertices. With EGC (e.g., $\theta = 2$ and $iter = 0$), CHROMA additionally includes vertices with degree at most $M + \theta$ (highlighted in red). Instead of assigning the same priority to all selected vertices, EGC computes priorities using $iter + (d[v] - M)$, yielding distinct priorities within the selected set. The remaining vertex (gray) is prioritized in a subsequent iteration after M and $iter$ are advanced.

E. Robust Graph Partition (RGP)

We develop **Robust Graph Partition (RGP)** to distribute large-scale graphs across multiple GPUs, as illustrated in Fig. 2. Algorithm 7 outlines the RGP procedure. In the graph partitioning phase, RGP uses an external partitioner to split the input graph G into a set of subgraphs $\mathcal{G}$ and a boundary graph b . The boundary graph b contains boundary vertices that connect to multiple subgraphs. To enable multi-GPU execution, RGP first processes b on a single GPU to compute priorities b_p and colors b_c . It then assigns each subgraph $g \in \mathcal{G}$ to a GPU, computes priorities g_p , and colors the subgraph while respecting the pre-colored boundary b_c , producing g_c .

Finally, RGP merges the per-subgraph results $\{g_c\}$ to obtain the final coloring c .

Notably, before processing each subgraph $g \in \mathcal{G}$, RGP broadcasts the boundary colors b_c to the corresponding GPU. With b_c , each subgraph can account for cross-partition constraints at its boundary (e.g., when maintaining partial information for vertices adjacent to other partitions), improving the fidelity of priority allocation and reducing conflicts. RGP uses a ghost-vertex mechanism [16] that replicates one-hop neighbors from adjacent partitions to facilitate this broadcast. In addition, we also overlap the boundary graph processing (line: 2-3) on the first GPU with the subgraph preloading phase to hide the transfer latency of other GPUs.

Finally, RGP is partitioner-agnostic and can be readily adapted to different partitioners. In this paper, we evaluate CHROMA using two widely used partitioners, METIS [17] and KaHIP [18].

F. Ordering Consistency

To quantify the consistency of vertex priority rankings with respect to JP-SL^A, we employ a pairwise comparison approach [19]. We use this analysis to measure how closely our Smallest Degree Calibration (SDC) matches JP-SL^A and to compare against several JP-SL variants. Given two priority lists of length n , P_{SL} (JP-SL) and P_S (a JP-series variant), we define the total number of ordered vertex pairs:

$$T = n(n-1) \quad (1)$$

and the number of *concordant* pairs, i.e., pairs whose relative order agrees in both lists:

$$C = \sum_{i \neq j} \mathbf{1}[(P_{SL,i} - P_{SL,j})(P_{S,i} - P_{S,j}) > 0] \quad (2)$$

The consistency ratio between P_{SL} and P_S is then:

$$\text{Consistency Ratio} = \frac{C}{T} \quad (3)$$

This ratio quantifies the agreement in vertex priority orderings between the two lists. A higher ratio indicates closer agreement with the JP-SL^A ordering. We will evaluate the consistency of CHROMA with other ordering heuristics in Section VI-C.

V. PREDICTION OF ELASTIC NUMBER

Because the elastic parameter θ introduces a trade-off between coloring quality and execution time, selecting an appropriate value can be challenging. To address this issue, we employ a machine learning model to predict the maximum feasible θ for a given graph G , subject to the constraint that the resulting color count does not exceed that of the baseline configuration ($\theta = 0$). We define:

$$\theta^*(G) = \sup\{\theta \mid c(G, \theta) \leq c_0(G)\}. \quad (4)$$

where $c(G, \theta)$ denotes the number of colors used by CHROMA on G under elastic parameter θ , and $c_0(G) :=$

TABLE I: Graph Datasets Used for Evaluation

Graph name	Type	$ V $	$ E $
as-skitter	Internet topology	1,696,415	22,190,596
cit-Patents	Patent citations	3,774,768	33,037,894
delaunay_n24	Triangulation	16,777,216	100,663,202
Email-Enron	Communication	36,692	183,831
europe_osm	Road map	50,912,018	108,109,320
facebook	Social network	4,039	88,234
le450_25d	Leighton graph	450	50,000
r4-2e23	Random	8,388,608	67,108,846
school1	Class scheduling	385	19,095
Slashdot0811	Social network	77,360	905,468
Slashdot0902	Social network	82,168	948,464
soc-Epinions1	Social network	75,879	508,837
Stanford	Web links	281,903	2,312,497
twitter	Social network	81,306	1,768,149
wiki-Talk	Collaboration	2,394,385	5,021,410
wiki-Vote	Vote network	7,115	103,689
youtube	Social network	1,134,890	2,987,624
uk-2002	Web links	18,520,486	523,574,516

TABLE II: The configuration of CHROMA and its optimizations. The “v” indicates that the optimization is enabled.

Configuration	ELS	SDC	EGC
CHROMA	v		
CHROMA ⁺	v	v	
CHROMA*	v	v	v (predicted)

$c(G, 0)$. Rather than predicting runtime directly, we train a regression model $\hat{\theta}^*(G) \approx \theta^*(G)$.

Datasets and Model Training. We constructed the training dataset using all real-world undirected graphs available in SNAP [20], excluding those listed in Table I. To generate training data, we varied θ from 0 to 100 for each graph G_i and executed CHROMA to determine the maximum θ_i that satisfies condition 4. Because θ exhibits a long-tail distribution, we record θ_i only when $T(\theta_i - 1)/T(\theta_i) \geq 1.2$ to prevent θ from becoming excessively large, where $T(\theta_i)$ denotes the CHROMA runtime under θ_i . Each training sample was represented as a tuple $(|V_i|, |E_i|, \theta_i)$, where $|V_i|$ and $|E_i|$ denote the numbers of vertices and edges, respectively. We reserved 20% of the data for validation and evaluated the model on the datasets listed in Table I. We used linear regression, support vector regression, and random forest regression in Scikit-learn package with all default parameters to train the model. Since the elastic parameter is an integer, we round the predicted value to the nearest integer. To discourage overestimation (i.e., predicting $\theta'_i > \theta_i$), we use the violation ratio (VR) as the training loss:

$$VR = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(c(G_i, \theta'_i) > c_0(G_i)) \quad (5)$$

Training Results. The linear regression model performs the best among the three models. It has 3.66% VR and 0.26 MSE; while the support vector regression model has 4.19% VR and 0.23 MSE; and the random forest regression model has 6.02% VR and 0.3 MSE. This result shows that even the simple linear model can achieve good performance on this task; hence, we choose the linear model as our predictor. We also observed that

TABLE III: GPU vs CPU: Algorithm Comparison (times rounded to 1 decimal). The CPU parallel configuration uses 192 threads. The last row reports the geometric mean of color count and execution-time speedup relative to the baseline JP-SL^M.

Dataset	CPU Sequential (ms)			CPU Parallel (ms)			GPU (ms)					
	DSatur	Greedy	JP-SL ^M	JP-SL ^A	ADG	SLL	cuSL	ADG	SLL	CHROMA*		
as-skitter	70/ 4055.3	71/ 176.8	67/ 1743.9	67/ 298.3	73/ 306.2	71/ 86.2	70/ 451.9	80/ 4.7	73/ 4.8	69/ 45.0		
cit-Patents	12/ 9184.4	14/ 423.8	13/ 5396.9	14/ 977.7	14/ 735.7	14/ 99.6	14/ 453.5	18/ 2.5	16/ 3.1	14/ 35.5		
delaunay_n24	6/16763.2	7/ 959.3	5/ 6521.8	5/ 4311.1	7/2220.2	7/186.4	7/1413.8	7/ 9.1	7/ 9.0	7/ 29.2		
Email-Enron	25/ 22.5	29/ 1.6	27/ 16.8	28/ 15.0	30/ 145.3	29/ 29.0	27/ 12.3	33/ 0.4	31/ 0.3	28/ 2.9		
europe_osm	4/ 40493.2	5/1481.1	4/ 15047.0	4/ 5500.5	5/5027.3	5/330.3	4/ 2580.5	5/ 11.4	5/ 11.4	4/ 21.5		
facebook	71/ 9.7	76/ 0.7	74/ 8.6	73/ 20.6	77/ 129.3	78/ 34.2	74/ 10.4	88/ 0.7	87/ 0.6	74/ 3.2		
le450_25d	29/ 2.4	30/ 0.1	31/ 0.7	30/ 10.3	31/ 146.0	31/ 26.8	31/ 3.8	32/ 0.5	32/ 0.4	31/ 2.0		
r4-2e23	5/ 30823.8	7/ 876.2	6/ 6644.6	6/3545.7	8/1238.1	7/150.1	7/ 25.6	8/ 4.8	8/ 4.8	7/ 29.8		
school1	20/ 4.7	32/ 0.1	15/ 0.8	14/ 19.4	34/ 133.3	32/ 48.9	14/ 3.5	34/ 0.5	38/ 0.5	15/ 2.1		
Slashdot0811	30/ 72.6	41/ 7.1	34/ 47.8	33/ 56.7	40/ 143.4	40/ 29.4	32/ 23.1	43/ 0.4	39/ 0.4	32/ 3.3		
Slashdot0902	32/ 80.4	41/ 7.4	32/ 48.8	34/ 25.0	40/ 130.5	40/ 37.4	33/ 24.5	43/ 0.5	39/ 0.4	34/ 3.5		
soc-Epinions1	30/ 57.3	44/ 5.1	34/ 33.8	34/ 23.4	43/ 138.8	41/ 27.4	34/ 26.4	46/ 0.9	43/ 0.8	34/ 3.7		
Stanford	62/ 300.6	64/ 28.3	61/ 243.1	61/ 89.7	62/ 166.6	64/ 34.6	61/ 98.5	65/ 0.8	61/ 0.9	61/ 11.9		
twitter	73/ 159.6	73/ 18.3	71/ 167.5	71/ 39.0	74/ 206.2	73/ 30.8	71/ 90.1	77/ 1.2	74/ 0.9	72/ 11.3		
wiki-Talk	51/ 1089.4	72/ 63.7	56/ 543.4	55/ 129.6	73/ 342.8	69/ 68.9	56/ 197.4	83/ 5.5	67/ 4.1	56/ 11.7		
wiki-Vote	24/ 14.7	27/ 0.6	27/ 5.5	28/ 14.9	28/ 163.9	28/ 32.6	27/ 9.5	32/ 0.4	32/ 0.4	26/ 2.8		
youtube	26/ 877.9	33/ 52.2	28/ 500.0	28/ 119.5	34/ 244.8	31/ 39.5	28/ 101.6	43/ 1.2	33/ 1.0	27/ 8.8		
GEOMEAN	0.98/ 0.49	1.18/ 8.65	1/ 1	1/ 1.27	1.20/ 0.46	1.17/ 2.55	1.03/ 2.40	1.31/101.66	1.23/110.56	1.03/ 17.44		

violations primarily occur on graphs for which the relationship between θ and the number of colors is non-monotonic. Further evaluations are presented in Section VI-C.

VI. EVALUATION

We first evaluate CHROMA’s performance in terms of coloring quality and execution time against state-of-the-art algorithms on CPUs and GPUs. We then validate the effectiveness of our ELS, SDC, and EGC optimizations. Finally, we evaluate the scalability of RGP on a multi-GPU system.

A. Experiment Setup

Experimental machines. For single-GPU experiments, we used an Intel Core i9-14900 system with 64 GB DRAM and an NVIDIA RTX 4090 GPU (CUDA 12.6, driver 560). For multi-GPU experiments, we used a system with an Intel Xeon Gold 6154 CPU and eight NVIDIA RTX 4090 GPUs, which are connected with PCIe 4.0 x16. For CPU baselines (sequential [21], [11] and parallel [12], [13]), we used an Amazon EC2 c7i.metal-48xl instance [22] with two 4th Gen Intel Xeon Scalable processors (192 threads total). The CPU was released in Q3 2023 and is priced at approximately USD 1,588 per CPU on eBay [23], comparable to the NVIDIA RTX 4090 (Q1 2023, USD 1,599) [24].

Datasets. Table I lists the graph datasets used for evaluation. We evaluate CHROMA on a variety of widely used graph datasets from several well-known collections [20], [25], [26], [27], [28].

Baselines. For CPU baselines, we selected DSatur [21], Greedy [21], and JP-SL^M [11] as sequential algorithms; while we implemented JP-SL^A (which uses the cuSL priority allocation scheme), JP-ADG, and JP-SLL with OpenMP at 192 threads.

For GPU baselines, we selected csrcolor [29] (implemented in cuSparse library [30]), data_pq [31], data_wlc [31], and

kokkos-VB_BIT [32] as SGR-based algorithms. For the MIS-based algorithms, we focused on JP-series algorithms with three ordering heuristics, including JP-ADG [13], JP-SLL [10], and ECL-GC-R [15], [14]. To ensure a fair comparison, we employed ECL-GC’s color allocation kernel in each GPU baselines. For JP-SLL, we set the granularity parameter $\gamma = 1$. All the code can be found in our open-source repository.

CHROMA configurations. Table II presents the configurations of CHROMA under different combinations of optimizations. For CHROMA*, we employ the model trained offline, as described in Section V.

Evaluating method. Since several coloring algorithms are randomized or otherwise non-deterministic, we ran each dataset five times and report the best outcome, which aligns with prior works [14], [15], [32].

B. Performance

CPU-GPU Performance Comparison. We want to validate the performance of CHROMA on GPUs against CPU baselines. The results are shown in Table III. We make three key observations. First, parallel JP-SL^A achieves coloring quality comparable to JP-SL^M, which supports our motivation for designing cuSL based on JP-SL^A on GPUs. Second, DSatur has best color quality on most datasets, while SL-based algorithms have close color quality to DSatur and even better on *as-skitter*, *Stanford*, and *twitter*. Third, CHROMA* achieves a total color count only slightly higher than SL, while running nearly $17.4\times$ and $7.3\times$ faster than JP-SL^A and cuSL, respectively. These results highlight the cost-performance advantage of GPUs for graph coloring.

Performance Comparison with GPU SOTA. We evaluate the execution time and coloring quality of CHROMA on a single-GPU platform and compare it against state-of-the-art GC algorithms. Table IV summarizes the results.

Overall, MIS-based graph coloring algorithms achieve better coloring quality than their SGR-based counterparts. Among

TABLE IV: Performance comparison of graph-coloring algorithms (entries are colors / time (ms)). The **bold** font represents the smallest number of colors used in a dataset across different algorithms. The last row shows the geometric mean in color count and execution time speedup compared to the baseline cuSL.

Dataset	SGR-based algorithms				MIS-based algorithms							
	csrcolor	data_pq	data_wlc	VB_BIT	cuSL	JP-ADG	JP-SLL	ECL-GC-R	CHROMA	CHROMA ⁺	CHROMA [*]	
as-skitter	335/ 66.1	83/ 47.8	98/ 58.8	109/ 292.3	70/ 451.9	81/ 40	71/ 23.7	70/ 10.8	86/ 16.7	68 / 75.9	69/ 45.0	
cit-Patents	143/ 27.3	15/ 3.1	16/ 1.7	24/ 9.1	14/ 453.5	18/ 4.9	15/ 4.2	13/ 5.0	17/ 5.2	13/ 143.3	12 / 35.5	
delaunay_n24	48/ 82.1	7/ 7.5	8/ 6.8	9/ 12.1	7/ 1413.8	8/ 14	7/ 10.1	6/ 10.6	8/ 12.2	5 / 474.8	7/ 29.2	
Email-Enron	153/ 11.3	33/ 3.0	48/ 3.6	50/ 11.6	27 / 12.3	39/ 2.7	31/ 2.3	29/ 2.3	39/ 2.2	28/ 3.7	28/ 2.9	
europe_osm	32/ 229.8	5/ 26.8	5/ 33.7	7/ 12.5	4 / 2580.5	5/ 25.0	5/ 115.7	4 / 22.5	5/ 21.6	4 / 72	4 / 21.5	
facebook	208/ 13.0	76/ 3.7	87/ 6.1	88/ 12.5	74 / 10.4	87/ 2.6	87/ 2.5	76/ 1.4	83/ 2.7	74 / 4.4	74 / 3.2	
le450_25d	126/ 9.8	30 / 1.0	37/ 3.9	37/ 7.2	31/ 3.8	36/ 2.0	32/ 1.7	30 / 1.0	38/ 1.9	31/ 2.2	31/ 2.0	
r4-2e23.sym	48/ 43.9	8/ 6.6	9/ 6.5	10/ 9.9	7/ 25.6	8/ 9.4	8/ 7.7	6/ 8.0	8/ 9.5	6 / 40.5	7/ 29.8	
school1	159/ 11.0	33/ 2.4	38/ 2.8	41/ 7.7	14 / 3.5	41/ 2.2	38/ 1.9	33/ 1.0	39/ 1.9	14 / 2.1	15/ 2.1	
Slashdot0811	175/ 12.4	44/ 2.5	54/ 3.2	59/ 14.9	32 / 23.1	47/ 3.1	39/ 2.1	40/ 1.3	44/ 2.4	33/ 4.5	32 / 3.3	
Slashdot0902	181/ 12.8	46/ 2.8	55/ 4.2	60/ 15.1	33 / 24.5	47/ 3.0	39/ 2.3	40/ 1.2	39/ 2.3	33 / 4.6	34/ 3.5	
soc-Epinions1	221/ 14.1	46/ 3.0	65/ 4.3	69/ 23.3	34 / 26.4	54/ 4.3	43/ 2.9	41/ 3.7	51/ 3.1	34 / 5.1	34 / 3.7	
Stanford	238/ 31.8	64/ 19.4	62/ 7.6	64/ 71.3	61 / 98.5	65/ 14.0	61 / 13.1	64/ 5.5	61/ 7	61 / 22.9	61 / 11.9	
twitter	233/ 15.0	72/ 6.2	82/ 6.4	81/ 24.6	71 / 90.1	77/ 4.7	74/ 3.0	73/ 2.8	80/ 3.4	71 / 22.3	72/ 11.3	
wiki-Talk	394/ 64.1	82/ 35.5	103/ 27.6	112/ 149.9	56 / 197.4	84/ 24.0	67/ 16.8	72/ 55.1	85/ 10.2	57/ 16.8	56 / 11.7	
wiki-Vote	175/ 11.6	29/ 1.4	41/ 2.6	44/ 8.5	27/ 9.5	38/ 2.5	32/ 2.0	27/ 1.2	40/ 2.1	27/ 3.1	26 / 2.8	
youtube	203/ 27.1	42/ 9.2	52/ 7.8	60/ 44.6	28/ 101.6	43/ 9.2	33/ 7.6	31/ 12.0	40/ 3.4	28/ 12.4	27 / 8.8	
GEOMEAN	5.85/ 2.33	1.23/ 10.00	1.46/ 9.09	1.62/ 2.86	1.00/ 1.00	1.35/ 9.09	1.19/ 11.11	1.10/ 14.12	1.32/ 12.50	0.97/ 4.00	1.00/ 7.14	

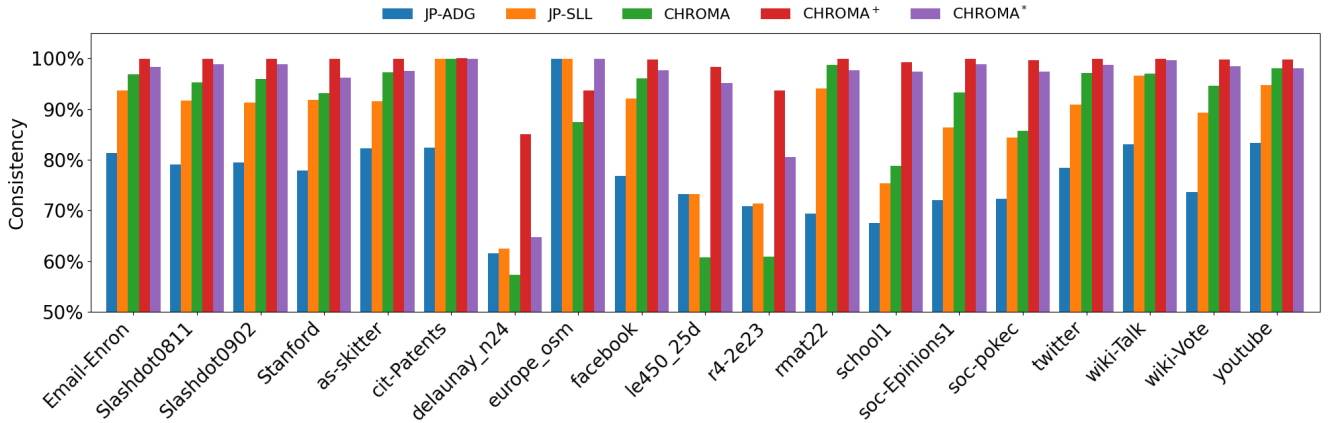


Fig. 5: Ordering consistency of approximate ordering heuristics and CHROMA with respect to JP-SL^A across various datasets.

them, cuSL provides the highest coloring quality, while ECL-GC-R offers the fastest execution time at the cost of reduced quality. Although the color reduction technique of ECL-GC-R can calibrate color quality of largest-first ordering at some level, cuSL still achieves much less color usage than ECL-GC-R in some graphs, such as *school1*, *Slashdot*, *soc-Epinions1*, and *wiki-Talk*. CHROMA achieves a 14 $\times$ geometric mean speedup over cuSL with ELS, despite the latter already being parallelized on GPUs. However, CHROMA incurs an average 1.35 $\times$ increase in the number of colors compared with cuSL due to the absence of smallest-degree calibration (SDC), which limits its ability to detect the global minimum degree. In contrast, CHROMA⁺ restores—and even enhances—the coloring quality, though its speedup decreases to 4.18 $\times$ relative to cuSL. By enabling the predicted elastic granularity control (EGC) optimization, CHROMA^{*} regains performance, achieving a 7.14 $\times$ speedup while maintaining coloring quality comparable to cuSL. These results show that CHROMA can effectively balance execution time and coloring quality via the predicted

elastic optimization strategy.

C. Ablation Studies

Impact of Smallest Degree Calibration. Fig. 5 reports consistency ratios mentioned in Section IV-F for JP-series algorithms. CHROMA shows lower consistency than JP-SL^A, while enabling Smallest Degree Calibration (SDC) yields the highest ratios on nearly all datasets. This indicates that SDC produces priority orderings that most closely match JP-SL^A, consistent with the analysis in Section IV-C. After enabling the learning-based Elastic Granularity Control (EGC), the consistency remains high and close to JP-SL^A, which helps explain why EGC maintains comparable coloring quality in Table IV.

Impact of Elastic Parameter θ . We evaluate how the elastic parameter θ affects iteration count, execution time, and coloring quality on three representative datasets. We also evaluate the performance of our predictor (Fig. 6). As θ increases, the number of iterations decreases rapidly at first,

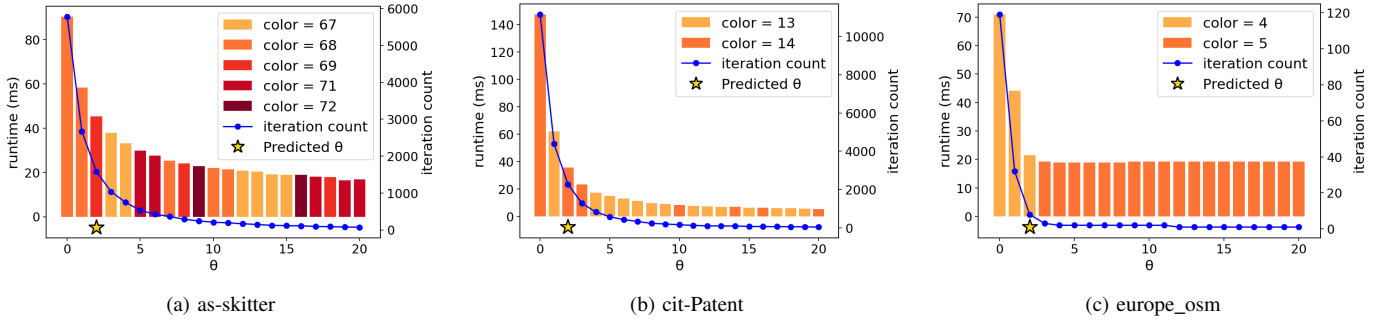


Fig. 6: CHROMA* runtime, color count, and iteration count across different values of θ . The x-axis indicates θ . The left y-axis shows execution time (bars), and the right y-axis shows iteration count (line). Different colors indicate the number of colors used. The 'star' denotes the predicted θ from the learning-based model.

with diminishing returns for larger values of θ , indicating that a relatively small θ can yield substantial runtime improvements. Although larger θ generally correlates with an increased number of colors, this relationship is not strictly monotonic across all graphs. In particular, higher-average-degree graphs (e.g., *as-skitter*, average degree 13.1) exhibit larger fluctuations, whereas lower-average-degree graphs (e.g., *cit-Patents*, 2.3; *europe_osm*, 2.1) behave more consistently. For example, *as-skitter* uses the most colors at $\theta = 9$ but the fewest at $\theta = 11$. This non-monotonicity can affect our conservative predictor, leading to slight quality degradation; more expressive models (e.g., graph neural networks) or additional structural features may further improve prediction accuracy.

D. CHROMA on Multi-GPU Systems

To evaluate the robustness of graph partitioning and CHROMA's partitioner-agnostic property, we use OpenMP [33] to enable multi-GPU execution with two partitioners, METIS [17] and KaHIP [18].

Fig. 7 shows the runtime scalability of CHROMA* on up to eight RTX 4090 GPUs for two large datasets, *uk-2002* and *delaunay_n24*. For fairness, we exclude graph partitioning time. We report the runtime in three phases: Phase 1 preloads subgraphs onto GPUs, initializes multi-GPU execution, and processes the boundary graph; Phase 2 performs priority allocation and color allocation for subgraphs on GPUs; Phase 3 uses multi-threading on the CPU to merge subgraph results into a final coloring.

In terms of coloring quality, *delaunay_n24* exhibits notable fluctuations, whereas *uk-2002* remains relatively stable as the number of partitions increases. This variation is driven by graph structure: *delaunay_n24* contains multiple disjoint communities, each with a single central vertex, and partitioning these communities tends to increase the overall color count.

Regarding runtime, *uk-2002* achieves a substantial speedup at two GPUs ($3.7\times$ over the single-GPU baseline), but the improvement diminishes as the GPU count increases. Although Phase 2 continues to decrease with additional GPUs under both METIS and KaHIP, the overheads of Phase 1 and Phase 3 increase and eventually offset the Phase 2 gains. This effect is amplified on *delaunay_n24*: since Phase 2 is only about

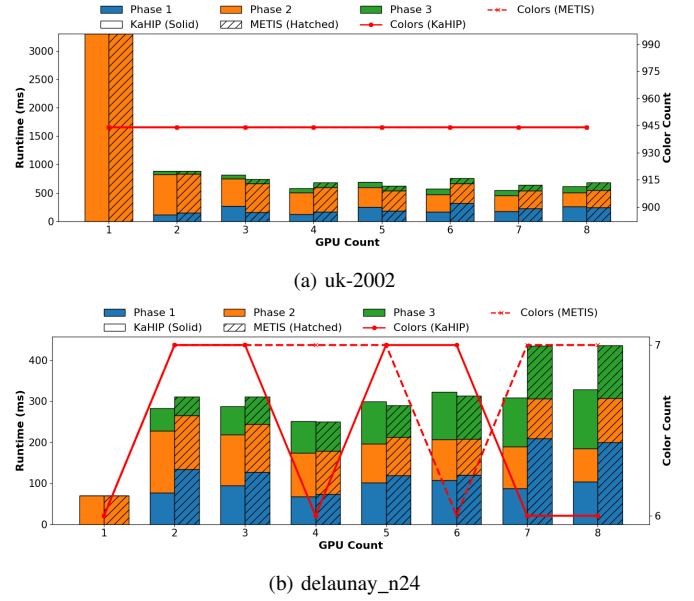


Fig. 7: Scalability analysis of CHROMA⁺ on a multi-GPU system using METIS (dashed) and KaHIP (solid) partitioners. The x-axis denotes the number of GPUs and the y-axis indicates runtime and color count. In *uk-2002* (a) the METIS color line overlaps with the KaHIP color line.

70 ms on a single GPU, additional GPUs primarily increase Phase 1/Phase 3 overhead, making the end-to-end runtime CPU-bound. We attribute the different scaling behaviors of *uk-2002* and *delaunay_n24* primarily to their degree distributions. Because PA runtime is strongly tied to the iteration count of the loop in Algorithm 3, the higher average degree of *uk-2002* makes its PA phase more time-consuming than that of *delaunay_n24*, which in turn affects multi-GPU scalability. On a single GPU, PA requires 92,679 iterations for *uk-2002* and 420 iterations for *delaunay_n24*. With two GPUs, the iteration count for *uk-2002* drops to 19,457 per GPU, whereas *delaunay_n24* remains about 411 per GPU. This also helps explain why Phase 2 stops improving beyond two GPUs: iteration-count reduction has largely plateaued for both graphs.

Overall, graphs with higher average degree or larger color counts are more likely to benefit from our RGP module.

VII. CONCLUSION

In this paper, we presented CHROMA, a high-quality and resilient graph coloring framework that enables efficient multi-GPU parallelization while maintaining coloring accuracy. CHROMA introduces cuSL, a GPU-based parallel priority allocation scheme for JP-SL. Built upon cuSL, CHROMA integrates three key optimizations to improve the efficiency and quality of the priority allocation process. For distributed execution, CHROMA achieves effective scalability across multiple GPUs through pre-colored boundary handling and ghost-vertex replication, exhibiting partitioner-agnostic behavior. Experimental results show that CHROMA attains up to a 7 $\times$ geometric mean speedup over state-of-the-art methods while preserving competitive color quality.

ACKNOWLEDGMENTS

This work was supported by the National Science Council of Taiwan under Grant NSC 112-2221-E-002-071. We thank Prof. Tsung-Wei Huang at University of Wisconsin-Madison for helpful discussions and the anonymous reviewers for their valuable comments.

REFERENCES

- [1] R. M. Karp, "Reducibility among combinatorial problems," in *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 2009, pp. 219–241.
- [2] G. J. Chaitin, "Register allocation & spilling via graph coloring," in *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction*, ser. SIGPLAN '82. New York, NY, USA: Association for Computing Machinery, 1982, p. 98–105. [Online]. Available: <https://doi.org/10.1145/800230.806984>
- [3] F. Wang, J. Liu, and E. F. Young, "Fastpass: Fast pin access analysis with incremental sat solving," in *Proceedings of the 2023 International Symposium on Physical Design*, 2023, pp. 9–16.
- [4] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your jacobian? graph coloring for computing derivatives," *SIAM Rev.*, vol. 47, pp. 629–705, 2005. [Online]. Available: <https://api.semanticscholar.org/CorpusID:8849423>
- [5] M. Naumov, P. Castonguay, and J. Cohen, "Parallel graph coloring with applications to the incomplete-lu factorization on the gpu," *Nvidia White Paper*, 2015.
- [6] U. Catalyurek, J. Feo, A. Gebremedhin, M. Halappanavar, and A. Pothen, "Graph coloring algorithms for multi-core and massively multithreaded architectures," 2012. [Online]. Available: <https://arxiv.org/abs/1205.3809>
- [7] T. Kaler, W. Hasenplaugh, T. B. Schardl, and C. E. Leiserson, "Executing dynamic data-graph computations deterministically using chromatic scheduling," in *Proceedings of the 26th ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 154–165. [Online]. Available: <https://doi.org/10.1145/2612669.2612673>
- [8] M. Luby, "A simple parallel algorithm for the maximal independent set problem," in *Proceedings of the seventeenth annual ACM symposium on Theory of computing*, 1985, pp. 1–10.
- [9] M. T. Jones and P. E. Plassmann, "A parallel graph coloring heuristic," *SIAM Journal on Scientific Computing*, vol. 14, no. 3, pp. 654–669, 1993. [Online]. Available: <https://doi.org/10.1137/0914041>
- [10] W. Hasenplaugh, T. Kaler, T. B. Schardl, and C. E. Leiserson, "Ordering heuristics for parallel graph coloring," in *Proceedings of the 26th ACM Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 166–177. [Online]. Available: <https://doi.org/10.1145/2612669.2612697>
- [11] D. W. Matula and L. L. Beck, "Smallest-last ordering and clustering and graph coloring algorithms," *J. ACM*, vol. 30, no. 3, p. 417–427, jul 1983. [Online]. Available: <https://doi.org/10.1145/2402.322385>
- [12] J. Allwright, R. Bordawekar, P. Coddington, K. Dincer, and C. Martin, "A comparison of parallel graph coloring algorithms," *SCCS-666*, pp. 1–19, 1995.
- [13] M. Besta, A. Carigiet, K. Janda, Z. Vonaarburg-Shmaria, L. Gianinazzi, and T. Hoefler, "High-performance parallel graph coloring with strong guarantees on work, depth, and quality," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '20. IEEE Press, 2020.
- [14] G. Alabandi, E. Powers, and M. Burtcher, "Increasing the parallelism of graph coloring via shortcutting," in *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 262–275. [Online]. Available: <https://doi.org/10.1145/3332466.3374519>
- [15] G. Alabandi and M. Burtcher, "Improving the speed and quality of parallel graph coloring," *ACM Transactions on Parallel Computing*, vol. 9, no. 3, pp. 1–35, 2022.
- [16] I. Bogle, E. G. Boman, K. Devine, S. Rajamanickam, and G. M. Slota, "Distributed memory graph coloring algorithms for multiple gpus," in *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*, 2020, pp. 54–62.
- [17] G. Karypis, "Metis: Serial graph partitioning and fill-reducing matrix ordering," <https://github.com/KarypisLab/METIS>.
- [18] P. Sanders and C. Schulz, "Think locally, act globally: Highly balanced graph partitioning," in *International Symposium on Experimental Algorithms*. Springer, 2013, pp. 164–175.
- [19] R. W. Saaty, "The analytic hierarchy process—what it is and how it is used," *Mathematical modelling*, vol. 9, no. 3-5, pp. 161–176, 1987.
- [20] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford large network dataset collection," <http://snap.stanford.edu/data>, Jun. 2014.
- [21] D. Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, vol. 22, no. 4, pp. 251–256, 1979.
- [22] Amazon. (2025) Amazon ec2 instance. Accessed: 2025-09-22. [Online]. Available: <https://aws.amazon.com/tw/ec2/instance-types/>
- [23] eBay. (2025) Intel xeon platinum 8488c cpu processor price on ebay. Accessed: 2025-09-22. [Online]. Available: <https://www.ebay.com/itm/176049544930>
- [24] NVIDIA. (2023) Rtx 4090 spec. Accessed: 2025-09-22. [Online]. Available: <https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/>
- [25] T. Davis and Y. Hu, "Suitesparse matrix collection," <https://sparse.tamu.edu/>, accessed: 2021-05-06.
- [26] M. Trick, "COLOR graph coloring instances," <https://mat.tepper.cmu.edu/COLOR/instances/>, 2018, carnegie Mellon University, accessed on 2025-03-31.
- [27] DIMACS, Center for Discrete Mathematics and Theoretical Computer Science, "Dimacs challenge graph instances," <http://www.dis.uniroma1.it/challenge9/download.shtml>, accessed: 2021-05-06.
- [28] Galois Project, ISS - The University of Texas at Austin, "Galois graph benchmark suite," <https://iss.odn.utexas.edu/?p=projects/galois>, accessed: 2021-05-06.
- [29] X. Chen and Li, "Csrcolor," <https://github.com/chenxuhao/csrcolor>, 2021, last accessed on May 6, 2021.
- [30] M. Naumov, L. Chien, P. Vandermersch, and U. Kapasi, "Cuspars library," in *GPU Technology Conference*, vol. 12, 2010.
- [31] X. Chen, P. Li, and C. Yang, "Efficient and high-quality sparse graph coloring on the GPU," *CoRR*, vol. abs/1606.06025, 2016. [Online]. Available: <http://arxiv.org/abs/1606.06025>
- [32] M. Deveci, E. G. Boman, K. D. Devine, and S. Rajamanickam, "Parallel graph coloring for manycore architectures," in *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2016, pp. 892–901.
- [33] C.-T. Yang, C.-L. Huang, and C.-F. Lin, "Hybrid cuda, openmp, and mpi parallel programming on multicore gpu clusters," *Computer Physics Communications*, vol. 182, no. 1, pp. 266–269, 2011.